

Доп. материал по основам ООП и не только

1. Явные конструкторы

Явный конструктор – конструктор, для которого запрещён неявный вызов. Объявляется с помощью ключевого слова `explicit`.

В строке 13 примера 1 приведён на первый взгляд очень странный код: объекту типа `MyClass` присваивается целочисленное значение. Данный код сработает: неявно будет вызван конструктор из строк 7-8. Поскольку у параметров есть значения по умолчанию, при вызове функции (в данном случае конструктора) эти параметры могут быть опущены. Присваиваемое значение попадает в первый параметр, остальные параметры получают значения по умолчанию.

Пример 1. Неявный вызов конструктора

```
1  class MyClass
2  {
3      int param1;
4      int param2;
5      int param3;
6  public:
7      MyClass(const int &value1=0, const int &value2=0, const int &value3=0)
8          : param1(value1), param2(value2), param3(value3) {}
9  };
10
11 int main()
12 {
13     MyClass obj = 11;
14     return 0;
15 }
```

В большинстве случаев такая запись является ошибкой. Но куда более частая ошибка – это путаница с апострофами и кавычками у строк ☺

Пример 2. Неявный вызов конструктора

```
1  #include <iostream>
2  #include <string>
3
4  class MyClass
5  {
6      std::string param;
7  public:
8      MyClass(const int &value = 0)
9      {
10         param.resize(value);
11     }
12     MyClass(char* const &str) : param(str) {}
13     void print()
14     {
15         std::cout << "str: " << param << "\nlen: " << param.length();
16     }
17 };
18
19 int main()
20 {
21     MyClass obj = 'a';
22     obj.print();
23     return 0;
24 }
```

Две маленькие чёрточки – и вот вместо строки «a» мы получаем пустую строку длины 97. А всё потому, что сработал не второй конструктор, а первый, и `char` преобразовался к целочисленному значению.

```
str:
len: 97
```

Чтобы такого не происходило, можно пометить конструктор как явный, используя ключевое слово `explicit`, тем самым запретив его неявное использование. Код из примера 3 уже не скомпилируется.

Пример 3. Явный конструктор

```
1  #include <iostream>
2  #include <string>
3
4  class MyClass
5  {
6      std::string param;
7  public:
8      explicit MyClass(const int &value = 0)
9      {
10         param.resize(value);
11     }
12     MyClass(char* const &str) : param(str) {}
13     void print()
14     {
15         std::cout << "str: " << param << "\nlen: " << param.length();
16     }
17 };
18
19 int main()
20 {
21     MyClass obj = 'a';
22     obj.print();
23     return 0;
24 }
```

2. inline-функции

inline-функции («встроенные функции») – функции, вызов которых заменяется компилятором на исходный код функции («тело»). Объявляются с помощью ключевого слова `inline` перед объявлением функции.

Пример 4. inline-функция

```
1  inline int getFist(const std::pair<int, std::string>& pair) const
2  {
3      return pair.first;
4  }
```

Плюс `inline`-функций в том, что их использование ускоряет работу программы, т.к. отсутствуют издержки на переход из одной функции в другую (разные функции лежат в разных областях памяти).

Минус inline-функций в том, что возникает дублирование кода, т.к. исходный код подставляется под каждый вызов. По этой причине «встраивать» рекомендуется только короткие функции, которые не сильно увеличивают код вызывающей их программы и издержки на вызов которых сопоставимы с временем выполнения самой функции.

Методы класса, определённые в заголовочном файле, являются встроенными сами по себе и не требуют ключевого слова `inline`.

3. Константные и статические члены классов

Поля и методы класса могут быть константными, т.е. неизменяемыми. Константные поля, как и обычные переменные, предваряются ключевым словом `const`. В случае объявления константного метода, `const` ставится в конце объявления функции (перед определением, если оно есть). Константные методы не могут менять значения полей класса.

Пример 5. Объявление константной функции

```
1 void print(const Vector& vector) const;
```

Классы также могут содержать статические поля и методы, «помеченные» ключевым словом `static`. Эти поля и методы существуют независимо от объектов класса и могут быть вызваны как через объект, так и через имя класса (с помощью оператора разрешения области видимости). Статические методы не могут менять поля класса, т.к. существуют независимо от объекта. В примере 6 приведён совершенно искусственный, зато лаконичный и наглядный пример использования статического поля в качестве счётчиков объектов типа `SomeObject`.

Пример 6. Использование статического поля класса

```
1  #include <iostream>
2  #include <vector>
3
4  class SomeObject
5  {
6  public:
7      static size_t counter;
8      SomeObject()
9      {
10         ++counter;
11     }
12 };
13
14 size_t SomeObject::counter = 0;
15
16 int main()
17 {
18     std::vector<SomeObject> vec(5);
19     for (size_t i = 0; i < 5; ++i)
20     {
21         std::cout << "Before " << SomeObject::counter << std::endl;
22         vec.push_back(SomeObject());
23         std::cout << "After: " << vec[0].counter << std::endl;
24     }
25     SomeObject a, b;
26     std::cout << "After + 2: " << vec[0].counter << std::endl;
27     return 0;
28 }
```

В строке 14 происходит повторное объявление `counter`, причём в глобальной области видимости. Всё в порядке, так и должно быть 😊

В цикле сначала выводится значение счётчика до добавления нового объекта, причём обращение идёт через имя класса, а после добавления обращения к счётчику происходит через нулевой элемент вектора. С таким же успехом можно обращаться к 1, 2, 3 и любому другому существующему на тот момент элементу вектора. После цикла создаются ещё 2 переменных типа `SomeObject`. Т.к. при создании переменных тоже вызывается конструктор, счётчик увеличится ещё на 2.

Ниже приведён результат выполнения программы. До первого вызова `push_back` в векторе уже лежат 5 объектов, т.к. они были созданы при вызове конструктора вектора с параметром 5 (строка 18).

```
Before 5
After: 6
Before 6
After: 7
Before 7
After: 8
Before 8
After: 9
Before 9
After: 10
After + 2: 12
```

4. Перегрузка операторов как членов класса

Большинство операторов возможно перегрузить как в качестве методов класса, так и в качестве внешних функций. При перегрузке бинарных операторов правый операнд передаётся как параметр функции, а **левым** выступает текущий объект.

В примере 7 приведён пример перегрузки оператора сложения для класса, описывающего точку на плоскости. В примере 8 приведён пример перегрузки этого же оператора, но уже как внешней функции. Использование оператора сложения будет выглядеть одинаково, и результат будет одинаковым. *Отмечу, что желание перегрузить оператор внешней функцией, слабый аргумент для того, чтобы делать поля публичными, и в данном примере «опубликование» применяется исключительно для простоты примера.*

Пример 7. Перегрузка оператора + как члена класса

```
1 class Point
2 {
3     float x;
4     float y;
5 public:
6     Point(const float& x_, const float& y_) : x(x_), y(y_) {}
7     Point operator+(const Point& other)
8     {
9         return Point(x + other.x, y + other.y);
10    }
11 };
```

Пример 8. Перегрузка оператора + как внешней функции

```
1 class Point
2 {
3 public:
4     float x;
5     float y;
6     Point(const float& x_, const float& y_) : x(x_), y(y_) {}
7 };
8
9 Point operator+(const Point& first, const Point& second)
10 {
11     return Point(first.x + second.x, first.y + second.y);
12 }
```

Есть случаи, когда перегрузка должна быть членом класса – это перегрузка оператора присваивания (=), скобок (как круглых, так и квадратных), оператора ->. Их нельзя перегрузить внешними функциями.

Иногда может требоваться несколько перегрузок одного оператора. Например, для точки из примеров выше можно перегрузить оператор сложения, принимающий в качестве правого параметра значение типа float, которое будет прибавляться к обеим координатам. Т.к. функции будут отличаться типами параметров, обе перегрузки будут рабочими. Но куда более частый вариант – это перегрузки, совпадающие по имени, типу данных и кол-ву параметров, но отличающиеся по константности. В примере 9 приведён как раз такой случай: присутствуют две перегрузки квадратных скобок – она неконстантная, а вторая константная. Первая нужна для модификации значений, полученных при обращении через квадратные скобки, а вторая – для того, чтобы была возможность оперировать квадратными скобками при работе с константным объектом (например, при передаче параметра в функцию по константной ссылке), т.к. константные объекты не могут использовать неконстантные методы.

Пример 9. Две перегрузки []

```
1 class Array
2 {
3     char* values;
4 public:
5     char& operator[](const size_t index)
6     {
7         return values[index];
8     }
9     const char& operator[](size_t& index) const
10    {
11        return values[index];
12    }
13 };
```

Несмотря на то, что все операторы, которые могут быть перегружены вне класса, могут перегружены и внутри него, в некоторых случаях перегрузку следует делать именно внешней функцией. Например, к таким случаям относятся перегрузки, в которых важен порядок операндов. Т.к. при перегрузке оператора внутри класса текущий объект выступает в качестве левого операнда, то можно перегрузить сложение как член класса Point для сложения Point и float, но для сложения float и Point уже потребуется перегрузка в виде внешней функции.

5. Дружественные функции и классы

Дружественные функции (friend-функции) – функции, имеющие доступ к полям класса с модификаторами `private` и `protected`, но не являющиеся членом класса. Объявляется внутри класса с ключевым словом `friend`. Может быть определена как внутри класса, так и вне его.

Дружественные функции возникают тогда, когда функциям, необходим доступ к полям класса, но они не могут стать членами класса (с точки зрения инкапсуляции или по другим соображениям). И этот механизм, как следует из определения, нарушает принципы инкапсуляции.

Дружественные функции не являются членами классов, а, следовательно, на них не распространяются модификаторы доступа.

Наиболее частый пример использования дружественных функций – это объявление перегрузки оператора `<<` для вывода на экран как дружественной функции. Поскольку слева от оператора должен стоять поток, этот оператор не стоит перегружать как член класса. А если у объекта много полей, которые нужно вывести на экран, при этом для многих из них не определены геттеры (ну нет в них необходимости в других случаях), может быть гораздо удобнее предоставить доступ к полям одной функции, а не добавлять кучу геттеров.

В примере 10 перегрузка `<<` объявлена как друг класса `Point`. Несмотря на то, что определена она вне класса, ей доступны приватные поля.

Пример 10. Перегрузка оператора `<<` для точки

```
1  #include <iostream>
2
3  class Point
4  {
5      float x;
6      float y;
7  public:
8      Point(const float& x_, const float& y_) : x(x_), y(y_) {}
9      Point operator+(const Point& other)
10     {
11         return Point(x + other.x, y + other.y);
12     }
13     friend std::ostream& operator<< (std::ostream& out, const Point& point);
14 };
15
16 std::ostream& operator<< (std::ostream& out, const Point& point)
17 {
18     out << "(" << point.x << "; " << point.y << ")";
19     return out;
20 }
21
22 int main()
23 {
24     std::cout << Point(0,1) + Point(2, 3);
25     return 0;
26 }
```

Аналогичным образом можно «подружить» класс с методами другого класса или класса целиком. Но не надо этим злоупотреблять, т.к. как уже было отмечено выше, это нарушает принцип инкапсуляции.

Пример 11. Дружественный класс и дружественный метод другого класса

```
1 class Drawer
2 {
3     void show();
4 };
5
6 class Figure
7 {
8     std::vector<Point> points;
9     friend class Point; // friend class
10    friend void Drawer::show(); // friend method
11 };
```

6. Функции

Функция (функциональный объект) – объект, для которого перегружен оператор (), что позволяет работать с ним, как с функцией.

В примере 12 два вектора сортируются с помощью функции `compFunction` и функтора `comp` соответственно. В первом случае третьим параметром в `std::sort` передаётся указатель на функцию (так функции можно передать в качестве параметров в другие функции, достаточно указать название функции), во втором – сам функтор.

Пример 12. Дружественный класс и дружественный метод другого класса

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 bool compFunction(const int& first, const int& second)
6 {
7     return first > second;
8 }
9
10 struct comp
11 {
12     bool operator()(const int& first, const int& second) const
13     {
14         return first > second;
15     }
16 };
17
18 int main()
19 {
20     std::vector<int> vec1{ 6, 0, 12, 1, 2, 9 },
21                     vec2(vec1);
22     std::sort(vec1.begin(), vec1.end(), compFunction);
23     std::for_each(vec1.cbegin(), vec1.cend(),
24         [](auto el) {std::cout << el << " "; });
25     std::sort(vec2.begin(), vec2.end(), comp());
26     std::cout << std::endl;
27     std::for_each(vec2.cbegin(), vec2.cend(),
28         [](auto el) {std::cout << el << " "; });
29     return 0;
30 }
```